GUIDE 4

Post-MD Analysis

Goal: Process the raw MD trajectory and extract structural, dynamic, and interaction-based information.

PHASE 1: Trajectory Preparation

1.1. Activate Environment

```
conda activate md_run
cd [PATH]/charmm-gui-[JOB_ID]/openmm
```

1.2. Process Trajectory (analyse.py)

This script loads all 10 DCD files, removes water/lipids/ions, re-images the system, aligns to the first frame, and saves a clean trajectory (analysis_result.dcd) and a topology PDB (analysis_result.pdb). These are used by all downstream analysis scripts.

```
python ~/PHD_MD/Scripts/silvia/analyse_silvia.py \
  -p step5_input.psf \
  -t step7_1.dcd step7_2.dcd step7_3.dcd step7_4.dcd step7_5.dcd \
     step7_6.dcd step7_7.dcd step7_8.dcd step7_9.dcd step7_10.dcd \
  -o analysis_result \
  --remove-waters
```

Output files:

- analysis_result.pdb — topology reference (first frame, no water/lipid)
- analysis_result.dcd — clean aligned trajectory, 10,000 frames

Note: If image_molecules fails with 'Could not find anchor molecules', this is a known mdtraj issue with PSF files. The script handles this automatically by skipping reimaging.

PHASE 2: Main Analysis Pipeline (run_analysis.py)

2.1. Configure run_analysis.py

Edit the Config section at the top of run_analysis.py before running. Key parameters to update for each new system:

```
SYS_NAME   = "holoMutant_protonated"   # label for output files
IS_HOLO    = True                       # True if ligand present
SIM_PATH   = "/path/to/openmm"          # directory with DCD files
PSF_FILE   = "step5_input.psf"
N_DCD      = 10
DT         = 0.05                       # ns per frame
OUTPUT_DIR = "/path/to/postMD_analysis"
RUN_MMGBSA = False                      # MM-GBSA done separately (see Guide
5)
```

2.2. Run the Pipeline

```
cd ~/PHD_MD/Scripts
python run_analysis.py
```

This runs 4 sub-scripts automatically:

- all-graphics.py — RMSF, protein RMSD (global + per-domain + P-loop), ligand distance maps, contact map
- clustering-dbscan.py — PCA + DBSCAN conformational clustering

- analyze_pore.py — pore dimensions at 3 levels (selectivity filter, drug binding pocket, intracellular gate)
- hbond_saltbridge.py — H-bond timeseries + K347 salt bridge occupancy

2.3. Output Files

- 01_protein_flexibility_rmsf.png — RMSF per residue
- 02_ligand_pore_distance.png — mexiletine distance to pore center over time
- 03_residue_contact_map.png — mexiletine contact map (4.5 Å cutoff) over time
- 04_ligand_conformational_rmsd.png — internal mexiletine RMSD
- 05_multisite_distances.png — mexiletine distance to K347, PRO348, pore center
- 06_ligand_density_map.dx — 3D density map of ligand positions
- 07_pore_dimensions_multilevel.png — pore distances at 3 levels
- 08_conformational_clustering_dbscan.png — PCA + DBSCAN cluster plot
- 09_protein_rmsd_domains.png — per-domain RMSD
- 10_protein_rmsd_global.png — global backbone RMSD
- 11_protein_rmsd_ploop.png — P-loop RMSD (resid 330–370)
- 12_hbond_saltbridge_summary.txt — numerical summary
- 13_hbond_timeseries.png — H-bond contacts over time
- 14_saltbridge_distance.png — K347–D356 distance over time
- 15_saltbridge_occupancy.png — salt bridge occupancy per residue

PHASE 3: HOLE Pore Analysis

HOLE calculates the pore radius profile along the channel axis across the full trajectory. Run separately from the main pipeline:

```
cd [SIM_PATH]
python ~/PHD_MD/Scripts/hole.py
```

Output:

- 06_HOLE_500ns_Average.png — mean pore radius ± SD with bottleneck annotation

Note: HOLE requires the hole executable to be installed and accessible in PATH.

PHASE 4: Contact Profile Analysis

Calculates per-residue contact frequency with mexiletine across all 10,000 frames using a 4.5 Å cutoff. Run interactively:

```
python3 << 'EOF'
import MDAnalysis as mda
import numpy as np
import glob

path = "/path/to/openmm"
dcds = sorted(glob.glob(f"{path}/step7_*.dcd"))
u = mda.Universe(f"{path}/step5_input.psf", dcds)

lig = u.select_atoms('resname UNL')
protein = u.select_atoms('protein')
contact_counts = {}
total = 0
```

```
for ts in u.trajectory:
    total += 1
    for res in protein.residues:
        d          =          mda.lib.distances.distance_array(lig.positions,
res.atoms.positions)
        if d.min() < 4.5:
            rid = res.resid
            contact_counts[rid] = contact_counts.get(rid, 0) + 1

print('Residue | Contact %')
for rid in sorted(contact_counts, key=contact_counts.get, reverse=True)[:20]:
    pct = contact_counts[rid] / total * 100
    print(f'  {rid:4d}  |  {pct:.1f}%')
EOF
```

PHASE 5: K347 Backbone vs Sidechain Contact

Distinguishes backbone vs sidechain contacts between mexiletine and K347:

```
python3 << 'EOF'
import MDAnalysis as mda
import glob

path = "/path/to/openmm"
dcds = sorted(glob.glob(f"{path}/step7_*.dcd"))
u = mda.Universe(f"{path}/step5_input.psf", dcds)

lig = u.select_atoms('resname UNL')
lys_sc = u.select_atoms('resid 347 and not (name N C CA O HA HN)')
lys_bb = u.select_atoms('resid 347 and (name N C CA O)')

total = contact_sc = contact_bb = 0
for ts in u.trajectory:
    total += 1
    d_sc = mda.lib.distances.distance_array(lig.positions, lys_sc.positions)
    d_bb = mda.lib.distances.distance_array(lig.positions, lys_bb.positions)
    if d_sc.min() < 4.5: contact_sc += 1
    if d_bb.min() < 4.5: contact_bb += 1

print(f'LYS347                sidechain:                {contact_sc}/{total}
({100*contact_sc/total:.1f}%)')
print(f'LYS347           backbone:                      {contact_bb}/{total}
({100*contact_bb/total:.1f}%)')
EOF
```

PHASE 6: ProLIF Interaction Fingerprint

Calculates a detailed interaction fingerprint (hydrophobic, H-bond, cationic, VdW, pi-stacking) between mexiletine and nearby residues across the trajectory:

```
python3 << 'EOF'
import MDAnalysis as mda
from MDAnalysis.topology import guessers
import prolif as plf
import glob

path = "/path/to/openmm"
```

```
dcds = sorted(glob.glob(f"{path}/step7_*.dcd"))
u = mda.Universe(f"{path}/step5_input.psf", dcds)

elements = guessers.guess_types(u.atoms.names)
u.add_TopologyAttr('elements', elements)

lig = u.select_atoms('resname UNL')
prot = u.select_atoms('protein and resid 340:380 1390:1440 1700:1730')

fp = plf.Fingerprint(['Hydrophobic','HBDonor','HBAcceptor',
                      'Cationic','VdWContact','PiStacking'])
fp.run(u.trajectory[::10], lig, prot)

df = fp.to_dataframe()
freq = df.mean() * 100
freq = freq[freq > 1.0].sort_values(ascending=False)
print(freq.to_string())
df.to_csv(f"{path}/prolif_output.csv")
EOF
```

Note: stride=10 (every 10th frame) is used to reduce computation time while maintaining representative sampling. Adjust the resid range in prot selection to match the binding region of your system.

PHASE 7: Representative Frame Selection - Silvia (Ligand-Filtered DBSCAN)

7.1. Prepare Clean Trajectory

Run analyse.py first (Phase 1) to generate analysis_result.dcd and analysis_result.pdb.

7.2. Configure and Run Clustering

Edit run_clustering_revised.py to set the ligand filter residues. For P-loop binding (neutral mexiletine):

```
ploop_resids=[348, 349, 353, 355]
```

For Domain III/IV binding (protonated mexiletine):

```
ploop_resids=[1714, 1423, 1424, 1400, 1420]
```

Then run:

```
cd ~/PHD_MD/Scripts/silvia
python run_clustering_revised.py
```

Output:

- rep_structure_full_10k.pdb — representative frame (full atoms including water/lipid)
- Trajectory index of the representative frame (for MM-GBSA, see Guide 5)
- Cluster statistics printed to terminal

Note: The ligand filter keeps only frames where mexiletine is within 5.0 Å of the specified residues. DBSCAN then clusters the filtered frames by backbone RMSD (eps=0.53 Å, min_samples=5). The medoid of the largest cluster is selected as the representative frame.